

anotações para a resolução do laboratório

Nícolas Rosenthal Dal Corso

Contents

1	Dificuldades e Erros	2
2	Modelo <i>Baseline</i>	2
2.1	Pré-Processamento dos Dados: Limpeza	2
2.1.1	Importação das Bibliotecas	2
2.1.2	Leitura dos Dados	2
2.1.3	Diferença na Quantidade de Colunas	3
2.1.4	Pipeline para Pré-Processamento	7
2.2	Pré-Processamento dos Dados: <i>Encoding</i> e <i>Scaling</i>	10
2.3	Definição do Modelo de Regressão Linear e Treinamento	11
2.4	Avaliação do Modelo <i>Baseline</i>	12
2.4.1	Resultados do Modelo <i>Baseline</i>	13
3	Modelo <i>stacking-ensemble</i> 4	13
3.1	Pré-processamento	13
3.1.1	Remoção de Outliers	13
3.1.2	Transformação da Variável Alvo	14
3.1.3	Imputação Contextual	14
3.1.4	Feature Engineering	14
3.2	Encoding de Variáveis Categóricas	15
3.3	Definição de Modelos Base	16
3.4	Validação Cruzada (Out-of-Fold)	16
3.5	Meta-Modelo (<i>Stacking</i>)	17
3.6	Resultados para o Modelo <i>stacking-ensemble</i> 4	17
3.6.1	Decisões de Design para o Modelo <i>stacking-ensemble</i> 4	17

1 Dificuldades e Erros

[modelo /baseline/] (n-rosenthal--submission-1) -> diversos modelos para testar e aprender técnicas avançadas -> [modelo /stacking-ensemble 4/] (n-rosenthal--submission-3)

No modelo *baseline*:

1. Após definir um modelo *baseline*, percebi que tratei a coluna Id como outra coluna qualquer;
2. Na primeira vez, eu defini duas estratégias de imputação de valores ausentes, uma depois da outra. Isto ocasionou uma *dupla imputação*. Isso não faz sentido nenhum.

2 Modelo *Baseline*

2.1 Pré-Processamento dos Dados: Limpeza

2.1.1 Importação das Bibliotecas

```
import numpy as np
import pandas as pd
```

2.1.2 Leitura dos Dados

Ler os dados a partir de documento .csv e inspeção de sua dimensionalidade. Também vamos armazenar o número de colunas em uma variável de trabalho.

```
train_df: pd.DataFrame = pd.read_csv(r"./lab05_files/lab05_files/train_student.csv");
test_df: pd.DataFrame = pd.read_csv(r"./lab05_files/lab05_files/test_student.csv");

print("Dimensões:");
print("\tdados de treinamento: ", train_df.shape);
print("\tdados de teste      : ", test_df.shape);

n_cols: int = test_df.shape[1];
```

Nosso objetivo é, a partir das 81 colunas, prever o valor da coluna SalesPrice. É observável também que os dados de teste possuem uma coluna a mais do que os dados de treinamento.

2.1.3 Diferença na Quantidade de Colunas

```
train_cols = set(train_df.columns)
test_cols = set(test_df.columns)

missing_in_test = train_cols - test_cols
extra_in_test = test_cols - train_cols

print("Colunas em `train`, mas não em `test` :", end="")
print(missing_in_test)
print("\nColunas em `test`, mas não em `train` :", end="")
print(extra_in_test)
```

1. Natureza das Colunas Por inspeção visual dos dados, observamos que existem colunas *numéricas* e colunas *categóricas*. Primeiro, vamos isolar a coluna que estamos interessados. Em seguida, vamos separar as colunas numéricas das colunas categóricas. É necessário verificar se estamos sendo capazes de extrair todas as colunas e se estamos separando-as adequadamente.

```
feature_cols = [col for col in test_df.columns if col != 'SalesPrice'] # test_d

numerical_cols = test_df[feature_cols].select_dtypes(include=['int64', 'float64'])
categorical_cols = test_df[feature_cols].select_dtypes(include=['object', 'categ

n_cols_numerical = len(numerical_cols)
n_cols_categorical = len(categorical_cols)

print("colunas numéricas : ", n_cols_numerical)
print("colunas categóricas: ", n_cols_categorical)
total = n_cols_numerical + n_cols_categorical
print("total : ", total, end="")
print(" -> ", total == n_cols) # n_cols = test_df.shape[1] = 80
```

Vamos primeiro analisar questões de limpeza dos dados para tanto colunas numéricas quanto categóricas. Em seguida, analisaremos os dois conjuntos separadamente e, quando formos propriamente treinar o modelo, reuniremos as colunas.

É muito provável que existam *valores ausentes* para algumas entradas. Vamos investigar:

```

# treinamento
missing_train = train_df.isnull().sum()
missing_train = missing_train[missing_train > 0].sort_values(ascending=False)

print("Valores ausentes nos dados de treinamento: ")
print(missing_train)
print(f"\ntotal: {missing_train.sum()}")

# teste
missing_test = test_df.isnull().sum()
missing_test = missing_test[missing_test > 0].sort_values(ascending=False)

print("\nValores ausentes nos dados de teste:")
print(missing_test)
print(f"\ntotal: {missing_test.sum()}")

```

Muitos valores faltando, tanto em colunas categóricas quanto numéricas, e tanto para os dados teste quanto para os dados de treinamento. A resolução deverá ser aplicada para ambos conjuntos (**train**, **test**) de forma análoga.

Existem algumas estratégias possíveis para lidar com este problema. Acaso uma coluna tenha mais do que cinquenta por cento de valores ausentes, será mesmo que ela é importante para a predição final?

```

# Fazer uma cópia dos DataFrames para evitar perder valores
cleaned_train_df: pd.DataFrame = train_df.copy();
cleaned_test_df: pd.DataFrame = test_df.copy();

# Definir threshold para eliminar colunas (>= 50%)
thresholds: list[float] = list(np.linspace(0.05, 0.99, num=20));
print(thresholds);

for threshold in thresholds:
    kill_cols_train = cleaned_train_df.columns[cleaned_train_df.isnull().mean() > threshold]
    kill_cols_test = cleaned_test_df.columns[cleaned_test_df.isnull().mean() > threshold]
    kill_cols: set = set(kill_cols_test) | set(kill_cols_train)

    print("threshold `", threshold, "`\n\t", list(kill_cols));

```

Definimos um intervalo linear de 20 valores entre 0.05 e 0.99 para os *thresholds* de forma a mostrar quais colunas possuem valores faltando. Percebemos que diversas colunas possuem pelo menos 5% dos valores ausentes: ['MasVnrType', 'GarageYrBlt', 'GarageFinish', 'PoolQC', 'Fence', 'GarageQual', 'Alley', 'MiscFeature', 'LotFrontage', 'FireplaceQu', 'GarageType', 'GarageCond']; este é um intervalo baixo e não podemos decidir nossa análise sobre ele. O outro limite da nossa análise (99% dos valores ausentes) apresenta apenas a coluna PoolQC.

Certamente não podemos esperar que uma coluna com valores ausentes para 99% das entradas seja importante para predição. Alguns thresholds interessantes são 0.25, 0.50, 0.75:

```
for threshold in [0.1, 0.25, 0.50, 0.75, 0.9]:
    kill_cols_train = cleaned_train_df.columns[cleaned_train_df.isnull().mean() > threshold]
    kill_cols_test = cleaned_test_df.columns[cleaned_test_df.isnull().mean() > threshold]
    kill_cols: set = set(kill_cols_test) | set(kill_cols_train)

    print("threshold `", threshold, "`\n", "\t".join(list(kill_cols)));
```

A decisão mais informada que tomamos é (1.) eliminar colunas com *muitos* valores ausentes e (2.) decidir manualmente a partir de inspeção mais profunda, para colunas com quantidade intermediária de valores faltando.

(a) Eliminar Colunas com Muitos Valores Ausentes

i. Decisão por coluna

```
missing_ratio_train = train_df.isnull().mean().sort_values(ascending=False)
missing_ratio_test = test_df.isnull().mean()

important_cols = ['MasVnrType', 'FireplaceQu', 'LotFrontage', 'GarageType']

print("Razões de ausência (train) - top 10:")
print(missing_ratio_train.head(10))
print("\nColunas que parecem ser importantes:")
for col in important_cols:
    if col in missing_ratio_train.index:
        print(f"{col}: train {missing_ratio_train[col]:.1%}, test {missing_ratio_test[col]:.1%}")
```

A. As colunas `PoolQC`, `MiscFeature` e `Alley` são altamente esparsas (≥ 0.93). Estas podem ser removidas sem detrimento à análise.

B. `Fence` é nosso *threshold* verdadeiro (0.8). A partir daqui, queremos que os valores ausentes sejam substituídos por uma categoria real (neste caso, ou `NoFence` ou `None` simplesmente).

C. As colunas posteriores permanecem, neste momento.

ii. Limpeza Propriamente Dita

```
cleaned_train = train_df.copy()
cleaned_test = test_df.copy()

# Colunas com muitos dados ausentes são eliminadas
cols_to_drop = ['PoolQC', 'MiscFeature', 'Alley', 'Fence']
cleaned_train = cleaned_train.drop(columns=cols_to_drop)
cleaned_test = cleaned_test.drop(columns=cols_to_drop)
```

(b) Preencher Colunas com Valores Ausentes Ainda existem algumas colunas com valores ausentes e precisamos que estes sejam preenchidos para podermos avançar para as próximas etapas. Abaixo, vamos analisar coluna-a-coluna

```
# 1. Imputar valores ausentes em categorias com novas categorias
cleaned_train['MasVnrType'] = cleaned_train['MasVnrType'].fillna('None')
cleaned_test['MasVnrType'] = cleaned_test['MasVnrType'].fillna('None')

cleaned_train['FireplaceQu'] = cleaned_train['FireplaceQu'].fillna('NoFireplac')
cleaned_test['FireplaceQu'] = cleaned_test['FireplaceQu'].fillna('NoFireplac')

# *Garagem*: se ausente, então o imóvel não tem garagem
garage_cols = ['GarageType', 'GarageFinish', 'GarageQual', 'GarageCond']
for col in garage_cols:
    cleaned_train[col] = cleaned_train[col].fillna('NoGarage')
    cleaned_test[col] = cleaned_test[col].fillna('NoGarage')

# *Subsolo/Porão*: se ausente, não há subsolo
basement_cols = ['BsmtQual', 'BsmtCond', 'BsmtExposure', 'BsmtFinType1', 'BsmtFinType2']
for col in basement_cols:
    cleaned_train[col] = cleaned_train[col].fillna('NoBasement')
    cleaned_test[col] = cleaned_test[col].fillna('NoBasement')
```

```

# 2. Imputação para valores numéricos: 0, mediana, moda
# LotFrontage: mediana, a fim de evitar outliers
#          alguns imóveis podem ter `Frontage` muito grande, ou nula
median_lot = cleaned_train['LotFrontage'].median()
cleaned_train['LotFrontage'] = cleaned_train['LotFrontage'].fillna(median_lot)
cleaned_test['LotFrontage'] = cleaned_test['LotFrontage'].fillna(median_lot)

# MasVnrArea: 0 (-> sem alvenaria)
cleaned_train['MasVnrArea'] = cleaned_train['MasVnrArea'].fillna(0)
cleaned_test['MasVnrArea'] = cleaned_test['MasVnrArea'].fillna(0)

# GarageYrBlt: 0 (-> sem garagem)
cleaned_train['GarageYrBlt'] = cleaned_train['GarageYrBlt'].fillna(0)
cleaned_test['GarageYrBlt'] = cleaned_test['GarageYrBlt'].fillna(0)

# Electrical: difícil entender o que isto significa
#          -> moda, valor mais comum
mode_elec = cleaned_train['Electrical'].mode()[0]
cleaned_train['Electrical'] = cleaned_train['Electrical'].fillna(mode_elec)
cleaned_test['Electrical'] = cleaned_test['Electrical'].fillna(mode_elec)

# 3. Inspeção: ainda há valores faltando?
print(f"Valores ausentes: train = {cleaned_train.isnull().sum().sum()}, test = {cleaned_test.isnull().sum().sum()}")
if cleaned_train.isnull().sum().sum() == 0:
    print("\tLimpeza completa, não há mais valores ausentes.")

```

2.1.4 Pipeline para Pré-Processamento

Agora que a análise manual foi feita, podemos **refatorar** o código acima para usar estratégias mais modernas com as *pipelines* do **sklearn**. Isto é interessante porque

1. é mais robusto e generalista;
2. elimina *warnings* do tipo `FutureWarning`; e
3. faz uso das técnicas vigentes em Ciência de Dados.

```

import pandas as pd
import numpy as np
from sklearn.impute import SimpleImputer

```

```

from sklearn.compose import ColumnTransformer

# Carregar os dados
train_df = pd.read_csv("./lab05_files/lab05_files/train_student.csv")
test_df = pd.read_csv("./lab05_files/lab05_files/test_student.csv")

print("1. Dados carregados. Dimensões:", train_df.shape, test_df.shape)

# Recomeçamos com cópias dos datasets iniciais
cleaned_train = train_df.copy()
cleaned_test = test_df.copy()

# 1. Eliminaremos colunas com >= 0.8 valores ausentes
cols_to_drop = ['PoolQC', 'MiscFeature', 'Alley', 'Fence']
cleaned_train.drop(columns=cols_to_drop, inplace=True)
cleaned_test.drop(columns=cols_to_drop, inplace=True)

print("2. Eliminadas colunas com >= 0.8 valores ausentes.")

# 2. Separar features (X) e target (y) para treinamento
X_train = cleaned_train.drop('SalePrice', axis=1)
y_train = cleaned_train['SalePrice']
X_test = cleaned_test

print(f"3. X_train dim.: {X_train.shape}")

# 3. Converter MSSubClass para string (categórica) antes da imputação
if 'MSSubClass' in X_train.columns:
    X_train['MSSubClass'] = X_train['MSSubClass'].astype(str)
    X_test['MSSubClass'] = X_test['MSSubClass'].astype(str)
    print("4. coluna `MSSubClass` convertida para `str`")

# 4. Definir colunas para imputação com zero (feature ausente = 0)
zero_impute_cols = [
    'MasVnrArea', 'GarageYrBlt', 'GarageCars', 'GarageArea',
    'BsmtFinSF1', 'BsmtFinSF2', 'BsmtUnfSF', 'TotalBsmtSF',
    'BsmtFullBath', 'BsmtHalfBath',
    'Fireplaces', 'PoolArea'

```

```

]
zero_impute_cols = [c for c in zero_impute_cols if c in X_train.columns]
print(f"5. Imputação de 0 para colunas numéricas com valores ausentes ({len(zero_imp

# 5. Colunas numéricas que receberão mediana
all_numerical = X_train.select_dtypes(include=['int64','float64']).columns.tolist()
median_impute_cols = [c for c in all_numerical if c not in zero_impute_cols and c !=
print(f"6. Imputação da mediana para colunas numéricas ({len(median_impute_cols)} col

# 6. Colunas categóricas
categorical_cols = X_train.select_dtypes(include=['object','category']).columns.tolis
print(f"7. Colunas categóricas ({len(categorical_cols)}): {categorical_cols[:5]}...)")

# 7. Criar imputers
num_imputer_median = SimpleImputer(strategy='median')
num_imputer_zero = SimpleImputer(strategy='constant', fill_value=0)
cat_imputer = SimpleImputer(strategy='constant', fill_value='Missing')

# 8. ColumnTransformer
preprocessor = ColumnTransformer([
    ('median_impute', num_imputer_median, median_impute_cols),
    ('zero_impute', num_imputer_zero, zero_impute_cols),
    ('cat_impute', cat_imputer, categorical_cols)
], remainder='passthrough')

print("8. Pré-processamento das colunas pelos `SimpleImputer`s")
X_train_imputed = preprocessor.fit_transform(X_train)
X_test_imputed = preprocessor.transform(X_test)

print("9. Imputação terminada.")

# 10. Verificação
print(f"X_train dim. após imputação: {X_train_imputed.shape}")
print(f"X_test dim. após imputação: {X_test_imputed.shape}")
print("Término da primeira fase do pré-processamento.")

1. Está Correto? Sim, está correto. O log mostra exatamente o que se
espera:
Dimensões:
Treino: (1022, 76) - pois de 81 colunas originais, removeu 4 (PoolQC,

```

MiscFeature, Alley, Fence) e removeu o target SalePrice $\rightarrow 81 - 4 - 1 = 76$.

Teste: (438, 76) - removeu as mesmas 4 colunas (teste não tem SalePrice) $\rightarrow 80 - 4 = 76$.

Divisão das colunas:

Zero-impute: 12 colunas (ex.: GarageYrBlt, MasVnrArea).

Mediana: 24 colunas (ex.: LotFrontage).

Catégoricas: 40 colunas (incluindo MSSubClass convertida para string).

Total: $12 + 24 + 40 = 76 \rightarrow$ consistente.

Não há valores ausentes após a imputação (o código que você mostrou não exibe mais np.isnan, então provavelmente removemos essa verificação que dava erro de tipo).

Portanto, a primeira fase do pré-processamento (imputação) está perfeita.

2.2 Pré-Processamento dos Dados: *Encoding e Scaling*

Antes de produzir o modelo de regressão propriamente dito, ainda estamos interessados em duas estratégias para adequar os dados ao problema. Queremos *codificar* (**encoding**) as colunas catégoricas e *escalar* (ou *normalizar*, **scaling**) as colunas numéricas.

```
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
import pandas as pd
import numpy as np

# 1. Verificação das listas de colunas e os DataFrames do bloco anterior
print("Colunas disponíveis :")
print(f"  median_impute_cols:  {len(median_impute_cols)}")
print(f"    zero_impute_cols:  {len(zero_impute_cols)}")
print(f"   categorical_cols:  {len(categorical_cols)}")
print(f"      total           :  {len(median_impute_cols) + len(zero_impute_cols) + len(categorical_cols)}")

# 2. Construir os pipelines para cada grupo e o transf. de colunas
```

```

numeric_median_pipe = Pipeline([
    ('impute_median', SimpleImputer(strategy='median')),
    ('scale', StandardScaler())
])

numeric_zero_pipe = Pipeline([
    ('impute_zero', SimpleImputer(strategy='constant', fill_value=0)),
    ('scale', StandardScaler())
])

categorical_pipe = Pipeline([
    ('impute_missing', SimpleImputer(strategy='constant', fill_value='Missing')),
    ('onehot', OneHotEncoder(handle_unknown='ignore', drop='first'))
])

# ColumnTransformer que aplica os três pipelines
preprocessor_final = ColumnTransformer([
    ('num_median', numeric_median_pipe, median_impute_cols),
    ('num_zero', numeric_zero_pipe, zero_impute_cols),
    ('cat', categorical_pipe, categorical_cols)
])

print("-" * 26);
print(preprocessor_final);

# 3. Transformar os dados
X_train_transformed = preprocessor_final.fit_transform(X_train, y_train)

print(X_train_transformed);

```

2.3 Definição do Modelo de Regressão Linear e Treinamento

A definição do modelo *baseline* é bastante simples, tratando-se apenas de regressão linear conforme disponibilizado pelo `sklearn`:

```

model = LinearRegression()

# Usando log-transform no target
y_train_log = np.log1p(y_train)

```

```
model.fit(X_train_transformed, y_train_log)
```

2.4 Avaliação do Modelo *Baseline*

```
y_pred_log = model.predict(X_train_transformed)
y_pred = np.exp1(y_pred_log)
```

```
rmse = np.sqrt(mean_squared_error(y_train, y_pred))
mae = mean_absolute_error(y_train, y_pred)
r2 = r2_score(y_train, y_pred)
rmse_log = np.sqrt(mean_squared_error(y_train_log, y_pred_log))
```

```
print("-" * 30)
print("11. Métricas no treino:")
print(f"RMSE (log): {rmse_log:.5f}")
print(f"RMSE: {rmse:.4f}")
print(f"MAE: {mae:.4f}")
print(f"R2: {r2:.4f}")
```

```
from sklearn.model_selection import cross_val_score
```

```
scores = -cross_val_score(
    model,
    X_train_transformed,
    y_train_log,
    scoring="neg_root_mean_squared_error",
    cv=5
)
```

```
print(f"CV RMSE (log): {scores.mean():.5f} ± {scores.std():.5f}")
```

```
y_test_pred_log = model.predict(X_test_transformed)
y_test_pred = np.exp1(y_test_pred_log)
```

```
print("12. Predições no conjunto de teste concluídas.")
```

```
submission = pd.DataFrame({
    "Id": test_df["Id"],
    "prediction": y_test_pred
})
```

```
submission.to_csv("submission.csv", index=False)
```

2.4.1 Resultados do Modelo *Baseline*

Métricas no treino:

- RMSE (log): 0.08926
- RMSE: 17406.9043
- MAE: 10889.9244
- R^2 : 0.9497
- CV RMSE (log): 0.18788 ± 0.05491

Em relação ao teste real, $\text{RMSE}(\log) \approx 0.23$

3 Modelo stacking-ensemble 4

`stacking-ensemble 4` é um **stacking ensemble** de dois níveis: cinco modelos de gradient boosting como camada base e um blend de RidgeCV + ElasticNet como meta-modelo.

3.1 Pré-processamento

3.1.1 Remoção de Outliers

Antes de qualquer modelagem, quatro filtros são aplicados ao conjunto de treino para eliminar observações atípicas que distorceriam o aprendizado:

```
train_df = train_df[train_df["GrLivArea"] < 4500]
train_df = train_df[~((train_df["GrLivArea"] > 4000) & (train_df["SalePrice"] < 20000))
train_df = train_df[train_df["LotArea"] < 100000]
train_df = train_df[train_df["TotalBsmtSF"] < 6000]
```

O segundo filtro é especialmente importante: remove casas grandes vendidas a preço baixo, que são anomalias conhecidas do dataset Ames e prejudicam modelos de árvore.

3.1.2 Transformação da Variável Alvo

O target `SalePrice` é transformado com `log1p` antes do treino:

```
y = np.log1p(train_df["SalePrice"])
```

Isso aproxima a distribuição da normal, reduz a influência de valores extremos e é o formato esperado na submissão — as predições são entregues em escala logarítmica, sem reverter com `expm1`.

3.1.3 Imputação Contextual

Em vez de imputação genérica por mediana, os valores ausentes são tratados com base no domínio do problema:

- **Catégoricas de ausência semântica** (ex: `GarageType`, `PoolQC`):
NaN

significa que o imóvel não possui aquela característica, portanto preenchidos com a string `"None"`.

- **Numéricas de área ou contagem** (ex: `GarageArea`, `BsmtFinSF1`):

imóveis sem garagem ou porão têm área zero, portanto preenchidos com 0.

- **Demais numéricas**: imputação por mediana dentro de cada fold de validação cruzada, usando `SimpleImputer`.

3.1.4 Feature Engineering

A função `add_features` expande o conjunto original de 80 para 103 features, agrupadas em quatro categorias:

1. Agregações de Área

```
df["TotalSF"] = df["TotalBsmtSF"] + df["1stFlrSF"] + df["2ndFlrSF"]
df["TotalPorchSF"] = df["OpenPorchSF"] + df["EnclosedPorch"] + ...
df["TotalBathrooms"] = df["FullBath"] + 0.5*df["HalfBath"] + ...
```

Capturam a área utilizável total, que é mais preditiva que qualquer área individual.

2. Flags Binárias de Presença

```

df["HasPool"]      = (df["PoolArea"] > 0).astype(int)
df["HasGarage"]    = (df["GarageArea"] > 0).astype(int)
df["HasBsmt"]      = (df["TotalBsmtSF"] > 0).astype(int)
df["HasFireplace"] = (df["Fireplaces"] > 0).astype(int)
df["IsRemodeled"]  = (df["YearRemodAdd"] != df["YearBuilt"]).astype(int)
df["IsNew"]        = (df["YrSold"] == df["YearBuilt"]).astype(int)

```

Modelos de árvore beneficiam-se de splits binários explícitos quando a distinção presença/ausência é mais relevante que a magnitude.

3. Interações Multiplicativas

```

df["OverallQual_TotalSF"] = df["OverallQual"] * df["TotalSF"]
df["OverallQual_GrLivArea"] = df["OverallQual"] * df["GrLivArea"]
df["OverallQual_sq"]       = df["OverallQual"] ** 2
df["QualCondScore"]        = df["OverallQual"] * df["OverallCond"]

```

`OverallQual` é a variável mais correlacionada com preço. Suas interações com área capturam o efeito não-linear de que qualidade importa proporcionalmente mais em casas grandes.

4. Transformações Logarítmicas e Temporais

```

df["LotAreaLog"] = np.log1p(df["LotArea"])
df["GrLivAreaLog"] = np.log1p(df["GrLivArea"])
df["HouseAge"] = df["YrSold"] - df["YearBuilt"]
df["RemodAge"] = df["YrSold"] - df["YearRemodAdd"]

```

Log-transforms reduzem a *skewness* de variáveis de área, ajudando especialmente o meta-modelo linear. As features de idade capturam depreciação e o efeito de reformas recentes.

3.2 Encoding de Variáveis Categóricas

As 44 variáveis categóricas são transformadas com **Target Encoding** dentro de cada fold:

```

enc = ce.TargetEncoder(cols=cat_cols, smoothing=10)
enc.fit(X_tr, y_tr)

```

O parâmetro `smoothing=10` é crítico: mistura a média da categoria com a média global proporcionalmente ao tamanho da categoria, evitando *data leakage* em categorias raras. O valor 10 (versus 0.3 do baseline) reduz significativamente o *overfitting* do *encoding*.

O encoder é sempre *fitado* apenas nos dados de treino do fold e aplicado em validação e teste, respeitando o isolamento de informação da validação cruzada.

3.3 Definição de Modelos Base

Cinco modelos de *gradient boosting* com hiperparâmetros distintos formam a camada base do *ensemble*. A diversidade entre eles é intencional para que erros não correlacionados se cancelam no stacking.

Modelo	n_estimators	learning_rate	num_leaves=/=depth	Característica
LGB1	6000	0.005	31	Conservador, balanceado
LGB2	8000	0.003	15	Raso, muito regularizado
LGB3	5000	0.008	63	Muitas folhas, agressivo
Cat	5000	0.007	6	Melhor OOF individual
XGB	5000	0.005	5	Complementa LGB/Cat

OOF RMSE individuais (seed 42):

- LGB1: 0.126874
- LGB2: 0.123090
- LGB3: 0.119819
- CatBoost: **0.116575** (melhor)
- XGBoost: 0.125426

3.4 Validação Cruzada (Out-of-Fold)

```
kf = KFold(n_splits=10, shuffle=True, random_state=42)
```

10 folds foram utilizados em vez dos 5 do baseline por dois motivos: cada modelo é avaliado em 10 conjuntos distintos, reduzindo a variância da estimativa de RMSE; e cada modelo de treino usa 90% dos dados em vez de 80%, gerando previsões mais fortes.

As previsões *Out-of-Fold* (OOF) são usadas como features para o meta-modelo, garantindo que o meta-modelo nunca veja previsões feitas sobre dados em que o modelo base foi treinado.

3.5 Meta-Modelo (*Stacking*)

As predições OOF dos cinco modelos base formam uma matriz de entrada para dois meta-modelos:

```
# Meta 1
meta_ridge = RidgeCV(alphas=np.logspace(-4, 2, 50), cv=10)
meta_ridge.fit(X_meta, y)    # RMSE: 0.113588

# Meta 2
meta_en = ElasticNet(alpha=0.0003, l1_ratio=0.5, max_iter=50000)
meta_en.fit(X_meta, y)      # RMSE: 0.114635

# Blend final (50/50)
final = 0.5 * meta_ridge.predict(...) + 0.5 * meta_en.predict(...)
# RMSE: 0.113965
```

RidgeCV seleciona automaticamente o melhor `alpha` entre 50 valores via *cross-validation* interna. O *blend* 50/50 com ElasticNet reduz ligeiramente a variância da predição final em relação a usar apenas um dos dois.

3.6 Resultados para o Modelo *stacking-ensemble* 4

- OOF RMSE FINAL (blend meta): 0.113965
- OOF RMSE FINAL (média direta): 0.119427

3.6.1 Decisões de Design para o Modelo *stacking-ensemble* 4

Decisão	Alternativa descartada	Motivo
Imputação contextual por domínio	Mediana global	NaN em GarageArea significa 0, não
RobustScaler	StandardScaler	Mais estável com outliers remanesce
TargetEncoding smoothing=10	smoothing=0.3	Menos leakage em categorias raras
10-fold CV	5-fold	Menor variância, mais dados por tre
5 modelos heterogêneos	Apenas LightGBM	Erros não correlacionados melhoram
RidgeCV como meta	ElasticNet puro	Seleção automática de alpha, mais e
Predição em log scale	<code>expm1</code> antes de submeter	Formato exigido pela competição